

Proven C Book

proven 라이브러리에 기반한 모던 C 입문

rubidus

차례

1	배경설명	5
1.1	프로그래밍이란 무엇인가	5
1.2	C는 어디에 있는가	5
1.3	지금 C의 세계는 요동치고 있다	5
1.4	왜 이 책을 만들었나	6
1.5	이 책을 읽는 법	7
2	단순한 기계 모델 ↔ C의 탄생	9
2.1	컴퓨터를 세 부품으로 그리기	9
2.2	비트 — 정보의 최소 단위	10
2.3	그 단순한 기계의 시절, C가 태어났다	11
3	워드와 주소 ↔ C의 원형	12
3.1	사물함 복도	12
3.2	워드 — 기계의 자연스러운 한 움큼	12
3.3	엔디안 — 여러 칸에 수를 넣는 두 가지 순서	13
3.4	C의 원형이 여기 있다	14
4	주소의 특수 지식 — 0번지, 정렬, 하위 비트	15
5	수의 표현 ↔ IEEE 754라는 계약	16
6	문자와 텍스트 ↔ 표준 속의 흉터	17
7	스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널	18
8	속도의 기계장치 ↔ 표준의 탄생	19
9	컴파일러 최적화 ↔ 추상 기계	20
10	그래서 C는 추상적인 언어다	21
11	헬로 월드	23
12	일반적인 컴파일 과정	24
13	개발환경 구축	25
14	프로그램의 구조	27
15	수식 — 값이 되는 것	28
16	함수 사용 — 부르는 법	29
17	출력	30
18	변수 선언	32
19	함수 선언과 정의	33
20	입력	34
21	정수 — 유한한 수의 세계	36
22	정수의 연산 — 나눗셈, 비트	37
23	불리언과 비교	38
24	결정 — if와 switch	39
25	반복 — 루프와 불변식	40
26	함수의 의미 — 값 복사와 부수효과	41
27	객체, 주소, 포인터	43
28	널 — 삼형제의 구별	44
29	포인터의 규칙 — 정렬과 프로버넌스	45
30	배열	46
31	문자열	47

32 안전한 입력과 proven의 등장	48
33 수명과 저장 기간	49
34 동적 메모리	50
35 구조체	52
36 공용체와 표현	53
37 실수 — 근사의 수학	55
38 오류와 계약	56
39 정의되지 않은 동작	57
40 여러 파일 — 분할과 링크	59
41 표준 라이브러리의 지형	60
42 proven 전면	61
43 모던 C 총정리	62

제1부 — 바탕

1 배경설명

이 장이 끝나면

프로그래밍이 무엇인지, C가 어떤 언어이고 지금 어떤 처지에 있는지, 그리고 왜 하필 지금 새로운 C 입문서가 필요한지 알게 된다. 이 책이 어떤 방식으로 쓰였고 어떻게 읽으면 되는지도 여기서 정한다.

1.1 프로그래밍이란 무엇인가

컴퓨터는 시키는 일을 하는 기계다. 문제는 시키는 방법이다. 컴퓨터는 사람의 말을 알아듣지 못하고, 지독하게 단순한 지시를 순서대로 받을 때만 움직인다. **프로그래밍**이란 우리가 원하는 일을 그 단순한 지시들의 목록으로 바꾸어 적는 일이고, 그 목록을 적는 약속된 표기법이 **프로그래밍 언어**다.

언어는 수백 가지가 있다. 그중 어떤 언어는 사람 쪽에 바짝 붙어 있어서 쓰기 편하고, 어떤 언어는 기계 쪽에 바짝 붙어 있어서 기계를 속속들이 다룰 수 있다. 이 책이 다루는 C는 후자의 대표다 — 반세기 동안 기계 쪽 자리를 지켜 온, 시스템의 언어다.

1.2 C는 어디에 있는가

C는 눈에 잘 띄지 않는다. 화려한 앱과 웹사이트의 얼굴에는 다른 언어들이 서 있다. 그러나 그 얼굴들 밑을 들추면 거의 어디에나 C가 있다.

● 오늘 하루가 C 위에서 흘러간다

아침에 스마트폰 알람이 울린다 — 그 운영체제의 핵심(커널)은 C로 쓰여 있다. 메시지를 보내면 통신 모듈의 펌웨어가 움직인다 — C다. 웹을 열면 서버와 브라우저가 데이터를 주고받는다 — 그 암호화 라이브러리와 통신 도구(OpenSSL, curl)가 C다. 앱이 무언가를 저장한다 — 세계에서 가장 널리 깔린 데이터베이스 SQLite가 C다. 자동차의 제동 장치, 세탁기의 제어판, 심박 조율기 — 임베디드 기기의 대부분이 C다. 파이썬으로 인공 지능을 돌려도, 그 밑에서 실제 계산을 하는 엔진의 상당 부분이 C(와 그 주변 언어)로 되어 있다. C를 배운다는 것은 이 보이지 않는 층을 읽고 쓸 수 있게 된다는 뜻이다.

1.3 지금 C의 세계는 요동치고 있다

그런데 왜 “지금” 새 입문서인가. 오래된 언어라면 오래된 책으로 배워도 되지 않는가. 그렇지 않다 — 지금 C를 둘러싼 풍경은 조용해 보여도 크게 요동치는 중이고, 이 요동이 이 책의 출발점이다.

첫째, C 안에서 여러 시대가 병립하고 있다. C는 표준이라는 공식 규격으로 관리되는 언어이고, C89(1989), C99, C11, C17을 거쳐 최신판 C23까지 왔다. 문제는 옛 표준이 은퇴하지 않는다는 것이다. 산업 현장에는 C89 시절 관행 그대로의 코드가 여전히 산더미처럼 살아 있고, 많은 교재가 그 시절 어법으로 쓰여 있으며, 그 곁에서 C23은 `bool`과 `nullptr` 같은 새 어휘를 표준에 들였다. 같은 “C”라는 이름 아래 서른 해 넘는 시차의 방언들이 공존한다.

문 옛 표준의 코드가 그렇게 많다면, 옛 어법부터 배우는 것이 실용적이지 않은가?

답 읽는 것과 쓰는 것을 나누면 답이 선다. 옛 코드를 읽을 일은 언젠가 생긴다 — 그래서 이 책도 역사와 옛 어법의 사연을 곳곳에서 다룬다. 그러나 새로 쓰는 코드가 옛 어법일 이유는 없다. 새 표준은 옛 표준의 함정을 수십 년 치 실전 경험으로 메운 결과물이다. 처음 배우는 사람일수록 가장 안전해진 오늘의 어법으로 시작하는 것이 이득이고, 옛 어법은 “왜 저렇게 썼는지”를 아는 교양으로 충분하다.

둘째, 이웃 언어들이 C의 영역을 잠식하고 있다. 한 지붕에서 출발한 C++은 빠른 주기로 판을 거듭하며 거대한 언어가 됐다 — 표현력은 강력해 졌지만, 언어 전체를 아는 사람이 드물다는 말이 나올 만큼 덩치가 불었다. 다른 쪽에서는 새 세대의 시스템 언어들이 나타났다. Rust는 C가 오래 겪어 온 메모리 사고를 언어 차원에서 봉쇄하겠다는 약속으로, Zig는 “C를 계승하되 더 단순하고 정직하게”라는 구호로, 각각 C가 지키던 자리를 파고 들고 있다. 운영체제 커널에 Rust 코드가 들어가기 시작한 것은 상징적인 사건이다.

셋째, 그 압박 속에서 C 자신도 변화의 조짐을 보이고 있다. C23은 근래 가장 큰 폭의 개정이었고, 표준 위원회는 포인터의 규칙을 더 엄밀하게 다듬는 작업(프로버넌스)과 안전성 논의를 이어 가고 있다. C는 느리게 움직이는 언어지만, 방향은 분명하다 — 더 명확한 계약, 더 안전한 기본값 쪽이다.

△ “C는 낡은 언어라서, 배워도 곧 쓸모없어진다”

그렇듯한 걱정이다 — 새 언어들이 저렇게 몰려오고 있으니. 그러나 현실의 수치는 반대를 가리킨다. 위에서 본 것처럼 세상의 기반 시설이 C 위에서 있고, 이 층은 교체 비용이 너무 커서 수십 년 단위로만 움직인다. 새 언어들조차 C와 대화하는 능력(C 인터페이스)을 생존 조건으로 갖추고 태어난다 — C는 시스템 세계의 공용어라서, Rust를 쓰는 Zig를 쓰는 결국 C를 읽고 이해해야 한다. 낡기는커녕, C는 이웃들이 늘어날수록 더 중요해지는 종류의 언어다.

문 그러면 차라리 Rust나 Zig부터 배우는 것이 낫지 않은가?

답 정직하게 답하면, 목적에 따라 다르다. 당장 안전한 응용 프로그램을 하나 만드는 것이 목표라면 다른 선택지도 좋다. 그러나 컴퓨터가 실제로 어떻게 움직이는지를 바닥부터 이해하는 것이 목표라면 C만 한 교재가 없다. C는 기계를 가장 얇게 감싼 언어라서, C를 배우는 과정이 곧 기계와 메모리와 컴파일러를 배우는 과정이 된다. 그리고 그 이해는 Rust로 가든 Zig로 가든 그대로 이고 갈 수 있는 밑천이다. 이 책은 그 밑천을 오늘의 어법으로 마련해 주는 책이다.

1.4 왜 이 책을 만들었나

이 요동치는 풍경이 이 책의 첫 번째 이유다. 여러 표준이 병립하고, 이웃 언어들이 경쟁을 걸어오고, C 자신이 변하고 있는 지금 — 옛 어법의 관성으로 쓰인 입문서가 아니라, **오늘의 C(C23)와 오늘의 관행에서 출발하는** 입문서가 필요하다고 판단했다. 그래서 이 책은 목차부터 다시 짰다. 문법 항목을 순서대로 나열하는 대신, 컴퓨터라는 기계의 기본부터 시작해 개념이 필요해지는 순서대로 C를 만난다.

두 번째 이유는 밝혀 두는 것이 정직하겠다. 저자는 **proven**이라는 C 라이브러리를 만들어 공개하고 있다. proven은 C 프로그래밍에서 가장 사고가 잦은 기본기 — 문자열 다루기, 입력 처리, 수의 변환 같은 일 — 를 검증된 형태로 제공하는 것을 목표로 하는 라이브러리다. C의 유명한 함정들은 대부분 이 기본기 층에서 터지는데, 표준 라이브러리는 역사적 이유로 그 함정을 그대로 안고 있다. 이 책의 뒷부분(제7부부터)에서는 그 함정이 실제로 왜 위험한지 보인 다음, proven이 그것을 어떻게 막는지 예제의 기반으로 사용한다. 이 책은 그 라이브러리를 알리려는 목적도 겸해서 만들어졌다 — 다만 순서는 지킨다. 먼저 표준 C만으로 개념을 온전히 배우고, proven은 그 필요가 스스로 증명된 시점에 등장한다.

문 라이브러리를 홍보하는 책이라면, 내용이 그쪽으로 치우치지 않는가?

답 치우치지 않도록 구조로 막아 두었다. 이 책의 앞 30여 개 장은 proven을 한 줄도 쓰지 않는다 — 순수하게 컴퓨터의 기본과 표준 C만으로 진행된다. proven이 등장하는 것은 “왜 이런 도구가 필요

한가”가 본문 속에서 스스로 드러난 뒤이고, 그때도 표준적인 방법을 먼저 보인 다음 비교로 제시한다. proven 없이도 이 책의 C 지식은 온전히 성립한다.

1.5 이 책을 읽는 법

이 책은 읽는 책이다. 당연한 말 같지만, 프로그래밍 책으로서는 드문 약속이다.

시키지 않는다. 이 책에는 연습문제가 없고, “직접 해 보라”는 지시도 없다. 컴퓨터 앞에 앉아 있지 않아도 된다 — 소파에서, 지하철에서, 그냥 소설처럼 앞에서 뒤로 읽으면 된다. 모든 코드는 지면 위에서 저자가 처음부터 끝까지 시연하고, 실행 결과까지 지면에 인쇄되어 있다. 그 실행 결과는 장식이 아니라 실제 결과다 — 이 책에 실린 모든 예제는 기계가 실제로 컴파일하고 실행해서 얻은 출력을 그대로 실는다.

문답으로 진행한다. 본문 곳곳에서 방금 읽은 내용에 대해 독자가 품었을 법한 질문을 그 자리에서 묻고, 바로 답한다. 질문을 만난 순간 잠깐 스스로 답을 떠올려 보고 다음 줄을 읽으면 가장 좋지만, 그냥 내리읽어도 된다 — 문답 자체가 설명의 일부다.

외우게 하지 않는다. 기억할 가치가 있는 것은 뒤 장의 서두에서 “돌아 보기”라는 이름으로 다시 찾아온다. 이전 장의 내용을 조금 더 깊은 질문으로 다시 만나게 되는데, 이것이 이 책의 복습 장치다. 잊었어도 괜찮다 — 답이 바로 옆에 있다.

작은 장으로 빠르게 간다. 장 하나는 짧다. 한 번에 하나의 착상만 다루고 다음으로 넘어간다. 큰 주제는 여러 장에 걸쳐 나선처럼 여러 번, 매번 조금 더 깊게 지나간다. 한 장에서 모든 것을 말하지 않으니, 어딘가 궁금증이 남았다면 그것은 대개 의도된 것이다 — 몇 장 뒤에서 그 궁금증을 다시 만나게 된다.

이제 시작한다. 첫걸음은 C가 아니라 컴퓨터 그 자체다 — C가 왜 이렇게 생겼는지는, C가 태어난 기계를 보아야 이해되기 때문이다. 다음 부에서 컴퓨터를 아주 단순한 그림으로 그리는 것부터 시작한다.

제2부 — 전산의 기본과 배경지식

이 부는 C 문법을 하나도 가르치지 않는다. 대신 이후의 모든 장이 딛고 설 바닥을 깎는다. 세 걸음으로 오른다.

기계와 기억 (2-4장) — 컴퓨터를 세 부품의 단순한 그림으로 그리고, 기억이라는 사물함 복도를 걷는다. 손에 잡히는 이야기다.

표현의 사다리 (5-7장) — 그 사물함에 수를, 글자를, 그리고 글자의 흐름을 담는 법. 한 단씩 오른다.

속도와 추상 (8-10장) — 기계가 복잡해지며 무슨 일이 벌어졌는지, 그래서 C가 왜 “추상적인 언어”인지. 이 부의 결론이 여기서 완성된다.

각 걸음마다 C의 역사가 나란히 걷는다. 연대와 인명을 외우게 하려는 것이 아니다 — 오늘의 C가 왜 이런 모양인지, 역사가 가장 짧은 설명이기 때문이다.

2 단순한 기계 모델 ↔ C의 탄생

이 장이 끝나면

컴퓨터를 세 가지 부품 — 계산하는 곳, 기억하는 곳, 박자를 맞추는 것 — 의 단순한 그림으로 그릴 수 있게 된다. 정보의 최소 단위인 비트가 무엇인지, 왜 하필 2진법인지 알게 된다. 그리고 그 단순한 기계의 시절에 C라는 언어가 어떻게 태어났는지 보게 된다.

돌아보기 1장에서 C가 운영체제와 임베디드 기기, 인터넷 기반시설 속에 지금도 살아 있다고 했다. 반세기가 넘은 언어가 어떻게 아직 그 자리에 있는가?

답 C가 서 있는 자리가 특별하기 때문이다. 그 자리는 기계와 사람이 만나는 가장 낮은 층이고, 기계는 반세기 동안 빨라졌을 뿐 일하는 방식의 뼈대는 놀랄 만큼 그대로다. 그 뼈대를 이 장에서 직접 들여다본다. 뼈대가 그대로인 한, 뼈대의 언어도 그대로 남는다.

2.1 컴퓨터를 세 부품으로 그리기

컴퓨터는 복잡한 물건이다. 그러나 복잡한 것을 이해하는 첫걸음은 언제나 과감하게 단순한 그림을 그리는 것이다. 지금부터 컴퓨터를 딱 세 부품으로 그린다.

첫째, **CPU**. 계산하는 곳이다. 더하고, 비교하고, 다음에 무엇을 할지 정한다.

둘째, **메모리**. 기억하는 곳이다. 계산에 쓸 재료와 계산의 결과가 여기에 놓인다.

셋째, **클럭**. 박자를 맞추는 것이다. 메트로놈처럼 일정한 간격으로 똑딱이고, CPU는 그 박자에 맞춰 한 걸음씩 일한다.

이 그림에서 컴퓨터가 하는 일은 이것이 전부다: **클럭이 똑딱일 때마다, CPU가 메모리에서 다음 할 일을 하나 가져와서, 그대로 한다.** 이것을 끝없이 반복한다.

문 “1초에 40억 번” 같은 광고 문구의 기가헤르츠(GHz)가 이 클럭인가?

답 그렇다. 4GHz는 메트로놈이 1초에 40억 번 똑딱인다는 뜻이다. 박자마다 CPU가 일을 한 걸음씩 진행하니, 박자가 빠를수록 같은 시간에 더 많은 걸음을 걷는다. 다만 걸음 수가 전부는 아니라는 것을 8장에서 보게 된다 — 한 걸음에 여러 일을 하는 요령이 현대 CPU의 진짜 무기다.

문 겨우 “가져와서 그대로 한다”의 반복으로 게임과 동영상과 인공지능이 어떻게 가능한가?

답 벽돌 한 장은 단순하지만 벽돌로 성을 쌓을 수 있는 것과 같다. 단순한 걸음이라도 1초에 수십억 번이면, 그리고 그 걸음들을 겹겹이 조직하면 — 작은 일들이 모여 함수가 되고, 함수가 모여 프로그램이 되고, 프로그램이 모여 운영체제가 되면 — 어떤 복잡한 일이든 지을 수 있다. 이 “겹겹이 쌓기”가 전산학의 심장인 **추상화**이고, 이 책 전체가 그 층계를 오르는 이야기다.

CPU가 “다음 할 일”로 가져오는 것을 **명령**이라 부른다. 명령은 대단한 것이 아니다. “이 수와 저 수를 더 해라”, “이 값을 저 칸에 넣어라”, “방금 결과가 0이면 다른 곳으로 건너뛰어라” — 이 정도 수준의, 지독하게 단순한 지시다. 프로그램이란 이런 명령을 순서대로 늘어놓은 목록이고, 프로그래밍이란 그 목록을 사람이 감당할 수 있는 방식으로 적는 일이다.

2.2 비트 — 정보의 최소 단위

메모리가 “기억하는 곳”이라 했다. 그러면 무엇을, 어떤 모양으로 기억하는가.

전자 회로가 가장 잘하는 일은 두 가지 상태를 구별하는 것이다. 전압이 높거나, 낮거나. 스위치가 켜져 있거나, 꺼져 있거나. 이 둘 중 하나를 담는 가장 작은 기억 단위를 **비트(bit)**라 부른다. 우리는 두 상태에 0과 1이라는 이름을 붙인다.

△ “컴퓨터 안 어딘가에 0과 1이 글자처럼 적혀 있다”

그렇듯한 생각이다 — 책과 화면마다 컴퓨터는 0과 1로 가득하다고 그려지니까. 그러나 기계 안에 숫자 글자가 있는 것이 아니다. 실제로 있는 것은 높거나 낮은 전압, 차 있거나 빈 전하 같은 물리 상태뿐이다. 0과 1은 그 물리 상태를 사람이 읽기 위해 붙인 **이름**, 즉 해석이다. 이 구별은 한가한 트집이 아니다 — “기억된 것 자체”와 “그것을 어떻게 읽을 것인가”의 분리는 이 책에서 계속 되돌아올 주제이고, 나중에 같은 비트 열을 수로도 글자로도 읽게 되는 이유다(5장, 6장).

문 왜 하필 두 상태인가? 스위치 하나에 0부터 9까지 열 가지 상태를 담으면 훨씬 효율적이지 않은가?

답 실제로 초기에는 10진 컴퓨터가 만들어지기도 했다. 그러나 열 단계의 전압을 구별하려면 단계 사이 간격이 좁아지고, 간격이 좁으면 잡음이나 온도 변화에 이웃 단계로 넘어가 버린다 — 즉 틀리기 쉽다. 두 상태는 간격이 가장 넓어서, 회로가 싸고 빠르고 잡음에 강하다. 2진법은 수학적 우아함 때문이 아니라 **공학적 정직함** 때문에 이겼다.

비트 하나는 두 가지를 구별한다. 비트를 여러 개 묶으면 구별할 수 있는 가짓수가 곱으로 불어난다. 두 개면 네 가지(00, 01, 10, 11), 세 개면 여덟 가지다. 여덟 개를 묶은 것에는 **바이트(byte)**라는 이름이 붙어 있고, 한 바이트는 256가지를 구별한다.

Σ 묶음의 크기와 가짓수

비트 n 개의 묶음이 구별할 수 있는 상태의 가짓수는, 각 자리가 독립적으로 두 값을 가지므로

$$2 \times 2 \times \cdots \times 2 = 2^n$$

이다. $2^8 = 256$, $2^{16} = 65536$, $2^{32} \approx 4.3 \times 10^9$. 이 책에서 2^n 은 계속 나타난다 — 기억의 크기, 표현 가능한 수의 범위, 주소의 개수가 전부 이 꼴이다.

묶음으로 수를 나타내는 방법은 우리가 쓰는 십진법과 같은 원리인 위치 기수법이다. 십진수 $304 = 3 \cdot 10^2 + 0 \cdot 10^1 + 4 \cdot 10^0$ 이듯, 2진수 $101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 5$ 다. 자리 하나가 10 대신 2의 거듭제곱을 짚어질 뿐이다.

문 256가지밖에 안 되는 바이트로 어떻게 큰 수를 다루는가?

답 십진법에서 한 자리로 부족하면 자리를 늘리듯, 바이트도 여러 개를 이어 붙여 큰 수를 담는다. 몇 개를 이어 붙이는 것이 자연스러운가는 기계마다 정해져 있다 — 그 “자연스러운 묶음”이 다음 장의 주인공인 워드다.

2.3 그 단순한 기계의 시절, C가 태어났다

지금까지 그린 세 부품의 그림은 오늘의 컴퓨터에게는 지나치게 단순한 그림이다(그 이야기가 8장이다). 그런데 1970년대 초의 컴퓨터에게는 이 그림이 거의 사실 그대로였다. 클록은 초당 수십만 번 수준이었고, CPU는 정말로 한 박자에 한 걸음씩 걸었고, 메모리는 정말로 한 줄로 늘어선 사물함이었다.

C는 바로 그 시절, 그 단순한 기계 위에서 태어났다. 벨 연구소의 연구자들이 PDP-11이라는 당시의 소형 컴퓨터에서 Unix라는 운영체제를 만들고 있었다. 운영체제는 기계를 구석구석 만져야 하는 프로그램이라, 그때까지는 기계마다 다른 어셈블리라는 저수준 표기로 짜는 것이 상식이었다. 그들은 어셈블리의 지루함을 덜려고 B라는 간결한 언어를 거쳐, 기계의 실제 모습 — 바이트로 된 메모리, 주소, 워드 — 을 언어의 개념으로 그대로 반영한 새 언어를 만들었다. 그것이 C다.

그래서 초기의 C는 과장을 조금 보태면 **그 시절 기계의 정직한 별명**이었다. C의 연산 하나가 기계 명령 한두 개에 그대로 대응했고, 프로그래머는 C를 쓰면서도 기계가 무엇을 할지 훤히 내다볼 수 있었다. C가 “기계에 가까운 언어”라는 평판은 여기서 왔다 — 그리고 이 평판이 오늘날 절반만 맞는 말이 된 사연이 이 부의 뒷장들(8-10장)이다.

● 이식성 혁명 — Unix를 C로 다시 쓰다

1973년경 Unix의 심장부가 어셈블리에서 C로 다시 쓰였다. 당시로서는 파격이었다 — 운영체제는 기계어 수준에서 짜야 한다는 것이 상식이었기 때문이다. 효과는 압도적이었다. 어셈블리 Unix는 PDP-11에 묶여 있었지만, C로 쓰인 Unix는 “C 컴파일러만 있으면” 다른 기계로 옮길 수 있었다. 운영체제가 특정 기계의 소유물이 아니게 된 것이다. 오늘날 Linux, Windows, macOS의 커널이 모두 C(또는 그 후예)로 쓰여 있고, 새 프로세서가 나오면 가장 먼저 C 컴파일러부터 이식되는 전통이 여기서 시작됐다.

문 50년 전 기계에 맞춘 언어를 지금 배우는 것이 손해는 아닌가?

답 거꾸로다. 기계의 뼈대 — 메모리, 주소, 바이트, 명령의 순차 실행 — 는 50년 동안 바뀌지 않았고, C는 그 뼈대를 가장 얇게 감싼 언어로 남아 있다. 그래서 C를 배우는 일은 특정 언어의 문법 암기가 아니라 컴퓨터 그 자체를 배우는 일에 가깝다. 이후에 어떤 언어를 쓰게 되든, 그 언어의 밑바닥에는 거의 언제나 C가 깔려 있다.

이 장의 그림을 정리하면 이렇다. 기계는 클록의 박자에 맞춰 메모리에서 명령을 가져와 그대로 실행하는 일을 반복하고, 기억은 두 상태의 최소 단위인 비트로 이루어진다. C는 이 그림이 거의 사실이던 시절에, 이 그림을 언어로 옮겨 태어났다.

다음 장은 세 부품 중 메모리를 자세히 들여다본다. 사물함 복도를 걸으며 칸마다 붙은 번호 — 주소 — 를 읽고, 기계가 한 번에 집는 자연스러운 묶음 — 워드 — 을 재고, 여러 바이트짜리 수를 사물함에 넣는 두 가지 순서 — 엔디안 — 를 구경한다. C의 포인터라는 유명한 개념이 바로 이 복도에서 태어났다.

3 워드와 주소 ↔ C의 원형

이 장이 끝나면

메모리를 번호 붙은 사물함들의 긴 복도로 그릴 수 있게 된다. 그 번호가 주소이고, 기계가 사물함을 한 번에 몇 칸씩 다루는지가 워드다. 여러 칸에 걸치는 수를 넣는 두 가지 순서 — 리틀 엔디안과 빅 엔디안 — 를 그림으로 구별하게 된다.

돌아보기 2장에서 바이트 하나는 256가지밖에 구별하지 못하며, 큰 수는 바이트를 여러 개 이어 붙여 담는다고 했다. 그러면 기계는 이어 붙인 바이트들이 “한 덩어리”라는 것을 어떻게 아는가?

답 모른다 — 이것이 이 장의 출발점이다. 메모리 자체는 어느 칸이 어느 칸과 한 덩어리인지 전혀 기록하지 않는다. 덩어리를 아는 것은 메모리가 아니라 **읽는 쪽**이다. 프로그램이 “여기서부터 네 칸을 하나의 수로 읽겠다”고 정할 뿐이다. 2장의 오개념 — 기억된 것 자체와 읽는 방법의 분리 — 이 여기서 첫 실전을 치른다.

3.1 사물함 복도

메모리를 그림으로 그리면 이렇다. 긴 복도에 같은 크기의 사물함이 한 줄로 끝없이 이어져 있고, 사물함마다 번호가 붙어 있다. 사물함 한 칸의 크기는 1바이트다. 번호는 0부터 시작해 1씩 늘어난다. 이 번호가 **주소(address)**다.

01001000	01101001	00100001	00000000	11111111
100	101	102	103	104

칸 안에 든 것은 언제나 그저 비트 여덟 개다. 그 여덟 비트가 수인지, 글자인지, 더 큰 무엇의 조각인지는 칸 자신도 복도도 모른다 — 읽는 쪽의 해석이 정한다.

주소에 관해 첫눈에 안 보이는 중요한 사실이 하나 있다. **주소 자체도 수다**. 사물함 번호는 그냥 정수이고, 정수이므로 그것을 다른 사물함에 넣어 둘 수도 있다. “347번 칸에, 512라는 번호를 적어 둔다” — 이상할 것이 하나도 없다. 이 평범한 착상이 나중에 C에서 가장 유명한 개념인 **포인터**가 된다(27장). 지금은 “주소도 수라서 어디에든 적어 둘 수 있다”까지만 챙기면 된다.

문 복도의 길이, 그러니까 사물함의 개수는 얼마나 되는가?

답 주소를 몇 비트로 적느냐가 정한다. 주소가 n 비트 수라면 사물함은 최대 2^n 개까지 구별할 수 있다(2장의 2^n 이 바로 다시 나왔다). 주소를 32비트로 적던 시절의 복도는 최대 약 43억 칸 — 4GiB — 이었고, 오늘의 64비트 주소는 사실상 다 쓸 수 없을 만큼 긴 복도를 만든다. “32비트 시스템에서 메모리 4GB 한계” 같은 말의 정체가 이것이다.

3.2 워드 — 기계의 자연스러운 한 움큼

사물함은 한 칸씩 있지만, 기계가 일할 때 한 칸씩만 집으면 너무 느리다. 그래서 CPU는 여러 칸을 **한 움큼**에 집도록 만들어져 있다. 그 움큼의 크기 — 기계가 가장 자연스럽게, 한 번에 다루는 비트 수 — 를 워드(word)라 부른다.

“64비트 컴퓨터”라는 말이 바로 이것이다. 그 기계의 워드가 64비트, 즉 8바이트라는 뜻이다. CPU 안의 임시 보관함(레지스터)이 그 크기이고, 메모리를 오가는 통로(버스)가 그 폭에 맞춰져 있고, 주소도 대

개 그 크기의 수로 다룬다. 워드는 기계의 “손 크기”라 해도 좋다 — 손이 큰 기계는 한 번에 더 큰 수를 집는다.

문 그러면 64비트 기계에서는 모든 것이 8바이트 단위로 저장되는가?

답 아니다. 저장은 여전히 바이트 단위로 자유롭다 — 1바이트짜리 글자도, 4바이트짜리 수도 같은 복도에 산다. 워드는 “기계가 가장 편하게 집는 크기”이지 “모든 것의 크기”가 아니다. 실제로 C에는 크기가 다른 여러 수 타입이 있고(21장), 어떤 크기의 데이터를 어디에 두는 것이 편한가라는 문제는 4장(정렬)에서 다시 만난다.

3.3 엔디안 — 여러 칸에 수를 넣는 두 가지 순서

이제 이 장의 하이라이트다. 4바이트짜리 수 하나를 사물함 네 칸에 넣어야 한다. 수를 16진법으로 12 34 56 78이라는 네 바이트 조각으로 나눴다고 하자(16진법 두 자리가 1바이트다). 100번 칸부터 넣는다면 — 어느 조각을 100번 칸에 넣어야 하는가?

두 학파가 있다. **빅 엔디안(big-endian)**은 큰 자리부터 넣는다. 사람이 종이에 수를 적는 순서 그대로다.

12	34	56	78
100	101	102	103

리틀 엔디안(little-endian)은 작은 자리부터 넣는다. 뒤집힌 것처럼 보이지만, “ i 번째 칸에 i 번째로 작은 자리가 있다”는 나름의 정연한 규칙이다.

78	56	34	12
100	101	102	103

지금 이 글을 읽는 컴퓨터와 스마트폰은 거의 확실히 리틀 엔디안이다(인텔·AMD·대부분의 ARM 운용). 날짜 표기와 똑같은 상황이다 — 2026-08-04라고 적는 나라와 04-08-2026이라고 적는 나라가 있고, 어느 쪽도 틀리지 않았지만, **서로의 편지를 그대로 읽으면 사고가 난다.**

문 사람 눈에는 빅 엔디안이 자연스러운데, 왜 대부분의 기계는 “뒤집힌” 리틀 엔디안을 택했는가? 무슨 장점이 있는가?

답 두 가지 실속 때문이다.

첫째, **시작 칸이 흔들리지 않는다.** 리틀 엔디안에서는 수의 가장 작은 자리가 언제나 시작 주소 그 칸에 있다. 위 그림의 수를 “4바이트 전부”로 읽든 “아래 2바이트만”·“아래 1바이트만”으로 줄여 읽든, 읽기 시작하는 칸은 똑같이 100번이다. 빅 엔디안에서는 읽는 폭을 바꿀 때마다 시작 칸을 옮겨 다시 계산해야 한다. 같은 값을 크기가 다른 눈으로 읽는 일이 잦은 기계에게 “시작 주소 불변”은 꽤 큰 단순함이다.

둘째, **계산의 진행 방향과 맞는다.** 손으로 덧셈을 할 때를 떠올리면 된다 — 일의 자리부터 더하고 받아올림을 위로 넘긴다. 여러 바이트짜리 수를 더하는 기계도 마찬가지로 작은 자리부터 처리하는데, 리틀 엔디안에서는 그것이 “낮은 주소부터 순서대로”와 정확히 일치한다. 초기의 작은 CPU들은 첫 바이트를 가져오자마자 덧셈을 시작할 수 있었고, 이 이점이 초창기 설계(인텔 계열의 조상들)에 스며서 오늘날까지 이어졌다.

빅 엔디안의 장점도 공정하게 적으면 — 덤프가 사람 눈에 그대로 읽히고, 바이트를 앞에서부터 통째로 비교하면 수의 대소 비교와 일치한다. 그래서 통신 규약처럼 “사람과 기계가 함께 들여다보는 자리”에는 빅 엔디안이 남았다. 어느 쪽도 우월하지 않다 — 자주 하는 일이 다를 뿐이다.

△ “메모리를 앞에서부터 읽으면 적힌 순서대로 수가 보인다”

그렇듯하다 — 종이에 적힌 수는 그렇게 읽으니까. 그러나 리틀 엔디안 기계에서 12 34 56 78이라는 수는 메모리에 78 56 34 12 순서로 놓인다. 메모리를 칸 순서대로 들여다보면(디버거나 파일 덤프에서 실제로 이렇게 본다) 수가 “뒤집혀” 보인다. 뒤집힌 것이 아니라, 넣는 순서의 약속이 다를 뿐이다. 기억할 것: 메모리 덤프는 종이에 적은 수가 아니다.

● NUXI — 순서가 낳은 유령

초기 Unix를 다른 회사의 컴퓨터로 이식하던 엔지니어들이 화면에서 기묘한 것을 보았다. UNIX라고 찍혀야 할 곳에 NUXI가 찍힌 것이다. 두 기계의 엔디안이 달라서, 2바이트 묶음 안의 글자 순서가 통째로 뒤집힌 결과였다. 이 사건은 “NUXI 문제”라는 이름으로 엔디안 사고의 대명사가 됐다. 같은 종류의 사고는 지금도 난다 — 파일을 한 기계에서 저장해 다른 기계에서 읽을 때, 네트워크로 수를 주고받을 때, 엔디안을 정하지 않으면 수가 깨진다. 그래서 인터넷 규약은 “통신할 때는 빅 엔디안으로 보낸다”는 **네트워크 바이트 순서**를 합의해 두었다.

문 내 컴퓨터가 리틀 엔디안이라는 것을 프로그램으로 확인할 수 있는가?

답 있다 — 여러 바이트짜리 수를 메모리에 넣고, 그 첫 칸을 1바이트만 따로 읽어 보면 된다. 첫 칸에 작은 자리 조각이 있으면 리틀 엔디안이다. 이 확인을 C로 실제로 해 보이는 것이 36장(공용체)의 시연이다 — 같은 기억을 다른 눈으로 읽는 도구가 그때 생긴다.

3.4 C의 원형이 여기 있다

이 장의 복도 그림은 C라는 언어의 설계도이기도 하다. C가 태어난 시절 (2장), 언어를 설계한 사람들은 기계의 이 모습을 감추는 대신 언어의 개념으로 그대로 들어 올렸다. 메모리는 바이트의 열이라는 것 — C의 모든 데이터는 바이트 단위 크기를 갖는다. 칸마다 주소가 있다는 것 — C에서는 거의 모든 것의 주소를 물을 수 있다. 주소도 수라는 것 — 그 수를 담는 타입(포인터)이 언어의 한복판에 있다. 다른 많은 언어가 “사물함 복도”를 프로그래머에게 보이지 않게 봉인해 둔 것과 대조적이다.

이 정직함은 양날이다. 기계를 훤히 내다보며 일할 수 있는 힘이자, 복도에서 길을 잃으면 그대로 사고가 되는 위험이다. 이 책의 제7부가 그 힘과 위험을 정면으로 다룬다.

다음 장은 이 복도의 구석에 있는 흥미로운 특례들을 구경한다. 0번 사물함에는 무엇이 있는가, 왜 두 칸짜리 짐은 아무 번호에나 못 넣는가(정렬), 그리고 번호의 끝자리가 항상 0이라는 사실을 이용해 몰래 정보를 숨기는 재주(태그 포인터)까지 — 주소의 세계가 생각보다 훨씬 사람 사는 동네 같다는 것을 보게 된다.

4 주소의 특수 지식 — 0번지, 정렬, 하위 비트

집필 예정 (RFC-0002 rev.d 4장).

5 수의 표현 ↔ IEEE 754라는 계약

잡필 예정 (RFC-0002 rev.d 5장).

6 문자와 텍스트 ↔ 표준 속의 흉터

집필 예정 (RFC-0002 rev.d 6장).

7 스트림의 기원 ↔ 펀치카드, 라인프린터, 프린터 터미널

잡필 예정 (RFC-0002 rev.d 7장).

8 속도의 기계장치 ↔ 표준의 탄생

집필 예정 (RFC-0002 rev.d 8장).

9 컴파일러 최적화 ↔ 추상 기계

잡필 예정 (RFC-0002 rev.d 9장).

10 그래서 C는 추상적인 언어다

집필 예정 (RFC-0002 rev.d 10장).

제3부 — 첫 프로그램

11 헬로 월드

집필 예정 (RFC-0002 rev.c 11장). 아래는 조판 파이프라인 검증용 표본이다 — demo 장치가 인쇄하는 실행 결과는 검증 스크립트가 실제로 실행해 캡처한 출력이다.

examples/ch11/hello.c

```
#include <stdio.h>

int main(void)
{
    printf("안녕, 세상!\n");
    return 0;
}
```

실행 결과

안녕, 세상!

12 일반적인 컴파일 과정

집필 예정 (RFC-0002 rev.c 12장).

13 개발환경 구축

집필 예정 (RFC-0002 rev.c 13장).

제4부 — 최소한의 도구 상자

14 프로그램의 구조

집필 예정 (RFC-0002 rev.c 14장).

15 수식 — 값이 되는 것

집필 예정 (RFC-0002 rev.c 15장).

16 함수 사용 — 부르는 법

집필 예정 (RFC-0002 rev.c 16장).

17 출력

집필 예정 (RFC-0002 rev.c 17장).

제5부 — 선언: 이름을 만드는 법

18 변수 선언

집필 예정 (RFC-0002 rev.c 18장).

19 함수 선언과 정의

집필 예정 (RFC-0002 rev.c 19장).

20 입력

집필 예정 (RFC-0002 rev.c 20장).

제6부 — 값과 흐름

21 정수 — 유한한 수의 세계

집필 예정 (RFC-0002 rev.c 21장).

22 정수의 연산 — 나눗셈, 비트

집필 예정 (RFC-0002 rev.c 22장).

23 불리언과 비교

집필 예정 (RFC-0002 rev.c 23장).

24 결정 — if와 switch

집필 예정 (RFC-0002 rev.c 24장).

25 반복 — 루프와 불변식

집필 예정 (RFC-0002 rev.c 25장).

26 함수의 의미 — 값 복사와 부수효과

집필 예정 (RFC-0002 rev.c 26장).

제7부 — 기억

27 객체, 주소, 포인터

집필 예정 (RFC-0002 rev.c 27장).

28 널 — 삼형제의 구별

집필 예정 (RFC-0002 rev.c 28장).

29 포인터의 규칙 — 정렬과 프로버넌스

집필 예정 (RFC-0002 rev.c 29장).

30 배열

집필 예정 (RFC-0002 rev.c 30장).

31 문자열

집필 예정 (RFC-0002 rev.c 31장).

32 안전한 입력과 proven의 등장

집필 예정 (RFC-0002 rev.c 32장).

33 수명과 저장 기간

집필 예정 (RFC-0002 rev.c 33장).

34 동적 메모리

집필 예정 (RFC-0002 rev.c 34장).

제8부 — 자료의 모양

35 구조체

집필 예정 (RFC-0002 rev.c 35장).

36 공용체와 표현

집필 예정 (RFC-0002 rev.c 36장).

제9부 — 정밀

37 실수 — 근사의 수학

집필 예정 (RFC-0002 rev.c 37장).

38 오류와 계약

집필 예정 (RFC-0002 rev.c 38장).

39 정의되지 않은 동작

집필 예정 (RFC-0002 rev.c 39장).

제10부 — 구성

40 여러 파일 — 분할과 링크

집필 예정 (RFC-0002 rev.c 40장).

41 표준 라이브러리의 지형

집필 예정 (RFC-0002 rev.c 41장).

42 proven 전면

집필 예정 (RFC-0002 rev.c 42장).

43 모던 C 총정리

집필 예정 (RFC-0002 rev.c 43장).